SpoilSense AI setup guide

Generic installation, deployment, and model generation procedure.

Purpose

This guide explains how to set up SpoilSense AI from source. The database and trained model files are created on the developer computer. They are not included in the uploaded source package.

1. Prepare the computer

Install Python 3.12 or newer, Node.js 22 or a current LTS release, and Docker only if container deployment is needed. Keep at least 8 GB of available memory for full model training.

2. Get the source

Copy or clone the project. Open a terminal in the folder containing backend, frontend, and inventory_dataset_1M.csv. The dataset is required for training.

3. Create and activate Python environment

Windows PowerShell:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

macOS or Linux:

python3 -m venv .venv

```
source .venv/bin/activate
```

4. Install dependencies

Windows:

```
pip install -r backend\requirements.txt
```

macOS or Linux:

```
pip install -r backend/requirements.txt
```

Then install frontend packages:

```
cd frontend
npm ci
cd ..
```

5. Generate the database

No database file is uploaded. The backend uses the built-in path backend/inventory.sqlite3. It creates this file, applies migrations from the source schema, creates indexes, and inserts the saved inventory and decay-history rows.

Start the backend once from the repository root. The first inventory data load creates the database automatically:

```
python -m uvicorn backend.main:app --host 0.0.0.0 --port 8000
```

The complete empty schema is available at backend/data/schema.sql. If SQLite is installed, an empty schema can also be created with:

```
sqlite3 backend\inventory.sqlite3 ".read backend\data\schema.sql"
```

The migration process remains responsible for inserting the saved inventory rows.

6. Train the model from scratch

No trained model files are uploaded. Run training after installing Python dependencies and confirming that inventory_dataset_1M.csv is present.

The program cleans the dataset, checks required columns, and splits the rows into 80 percent training, 10 percent validation, and 10 percent test data. Variety stratification keeps Banana, Orange, Pineapple, and Tomato represented.

The LightGBM branch learns from pH, moisture, temperature summaries, temperature slope, and variety features. The LSTM branch reads the five-day temperature sequence. The hybrid result combines the LightGBM prediction with an LSTM residual correction.

Run full training:

```
python -m backend.training.train_hybrid
```

For a quick installation check, run a small training job:

```
python -m backend.training.train_hybrid --max-rows 10000 --epochs 1
```

Training creates backend/models/lgbm_static.txt, hybrid_lstm.pt, scaler_stat.pkl, scaler_dyn.pkl, metadata.json, and metrics.json. These files are local generated artifacts and should not be uploaded.

7. Evaluate the model

Evaluation loads the newly created model files and checks them on held-out data. It reports mean absolute error, root mean squared error, and R-squared.

Run:

```
python -m backend.evaluation.evaluate_hybrid
```

The stored reference run used 1,000,000 rows. The hybrid result was approximately MAE 0.0665 days, RMSE 0.0834 days, and R-squared 0.999875. A new run can differ if the environment, library versions, dataset, or settings differ.

8. Run locally

Keep the backend terminal active:

```
python -m uvicorn backend.main:app --host 0.0.0.0 --port 8000
```

Open a second terminal:

```
cd frontend
npm run dev
```

Open the local browser address printed by the frontend development server.

9. Deploy with Docker

From the repository root:

```
docker compose up --build
```

Docker creates its own runtime database volume. Stop the services with:

```
docker compose down
```

For a clean rebuild, use docker compose down -v. This removes the Docker database volume and causes a new database to be created next time.

10. Upload rules

Upload source code, the dataset, dependency files, Docker files, schema.sql, README, guides, and screenshots.

Do not upload backend/inventory.sqlite3, any files under backend/models that are generated by training, .venv, frontend/node_modules, frontend/dist, Python caches, logs, or local environment files.

The next developer must run database setup and model training locally before using prediction features.

11. Troubleshooting

If prediction files are missing, run the full training command. If the database is missing, start the backend and load the application. If the frontend cannot load data, confirm both local processes are running. If training is slow, use the smaller test command first and then run full training.